Informe Final

Sistema implementado

Paquetes y clases: El código se organizó en `modelo` (subpaquetes `personas`, `mascotas`, `agenda`, `atencion`, `inventario`, `facturacion` con `pago`, y `enumeradores`), `controlador`, `servicio`, `persistencia` y `vistas` (`interfaces`, `consola`, `swing`); unas 7.800 líneas de Java en 70 archivos. Los puntos de entrada son `AppSwing` y `AppConsola`.

Arquitectura Modelo-Vista-Controlador (MVC): El sistema se organizó siguiendo este patrón arquitectónico, que separa la lógica de negocio de la interfaz de usuario:

Componente	Responsabilidad	Paquetes y clases
Modelo	Entidades y reglas de negocio (turnos, historia clínica, stock FEFO, facturación, etc.); no conoce ninguna vista. Incluye los servicios de dominio y la persistencia.	`modelo.*`, `servicio`, `persistencia`
Vista	Presenta los datos y recoge las entradas del usuario. Hay una interfaz por rol con dos implementaciones intercambiables: consola y Swing. No contiene reglas de negocio.	`vistas.interfaces`, `vistas.consola`, `vistas.swing`
Controlador	Recibe las acciones de la vista, valida los parámetros, delega en el modelo, persiste el estado y devuelve el resultado. Uno por familia de casos de uso.	`ControladorRegistro`, `ControladorTurnos`, `ControladorAtencion`, `ControladorFacturacion`, `ControladorAdministracion`

El flujo es siempre Vista → Controlador → Modelo, y el resultado vuelve por el mismo camino. Como las vistas solo dependen de los controladores (que reciben identificadores y documentos, no objetos del dominio), se pudo ofrecer dos interfaces sobre la misma lógica sin duplicarla, y las pruebas funcionales ejercitan los controladores directamente. Este esquema se corresponde con el patrón GRASP Controlador.

Persistencia: `EstadoSistema` agrupa clínica, agenda, inventario, facturas y tarifario; `RepositorioArchivo` lo serializa a `vetgo.dat` tras cada operación exitosa.

Cambios respecto del diseño original

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
Persistencia	`Repositorio<T>` genérico y `RepositorioArchivo<T>` (una lista por tipo)	`Repositorio` no genérico que guarda y carga un único `EstadoSistema` con todo el grafo	Simplificar el guardado de objetos y evitar referencias rotas entre archivos

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
Notificación	`NotificadorLog` (escribe en archivo)	`NotificadorConsola` (imprime por consola)	S-02 admite consola o log; la consola es más visible para la demostración
Tipos de atención	Consulta general, vacunación, cirugía; `Turno.urgencia: boolean`	Se agrega `TipoAtencion.URGENCIA` (30 min, con precio en el tarifario) y se elimina el booleano	Un solo atributo expresa el tipo y la duración, y la urgencia se factura con su propia tarifa
Estados	`EstadoTurno` sin «en curso»; `EstadoConsulta`: en curso / confirmada	Se agregan `EstadoTurno.EN_CURSO` y `EstadoConsulta.FACTURADA`	El turno refleja la consulta abierta; la consulta facturada no puede facturarse dos veces y deja de listarse como pendiente
Controladores	Recibían objetos del dominio	Reciben identificadores (`String`): id de mascota/turno/consulta, documento de dueño/veterinario	Desacoplar totalmente las vistas del modelo
Vistas	No estaban modeladas	Interfaces por rol con implementación de consola y Swing, y ventana de selección de rol	Adaptar el TP a la arquitectura MVC vista en clase y dar una forma al usuario de interactuar con el sistema
Facturación	`Factura.agregarLinea` crea las líneas; `calcularTotal()` público	Las líneas se arman en `ControladorFacturacion` y se pasan al constructor; el total se calcula al emitir y queda guardado; segundo constructor para facturas sin consulta (cargo por cancelación, seña)	La factura debe ser inmutable desde su emisión (RN-F4). El anterior diseño no lo aseguraba
Anulación	`Factura.anular` devuelve la `NotaCredito`	`anular(motivo, emisor)` deja la nota asociada a la factura	La nota pertenece a la factura anulada
Seña	Atributos y reglas en `Duenio`	Igual, más `ControladorFacturacion.cobrarSenia` que emite la factura de la seña; `pagarSenia` también reinicia el contador de ausencias; asistir a una consulta también lo reinicia	Cerrar el ciclo completo de señas
Urgencias	`atenderUrgencia(mascota, veterinario, día)`: primer horario del veterinario de guardia	`atenderUrgencia(mascota, veterinario)`: comienza ahora o cuando termina el turno en curso; desplaza en cascada y, si el turno desplazado no entra en	Es la interpretación más realista de «urgencia» en un sistema en uso

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
		la jornada, lo pasa a la próxima apertura (`Veterinario.proximaApertura`)	
Vacuna aplicada	`aplicarVacuna(vacuna, numeroLote, fecha)`	`aplicarVacuna(vacuna)`: toma el primer lote vigente (el que vence antes) y la fecha de hoy	Evitar que el veterinario cargue el lote a mano y garantizar FEFO
Identificadores	`numero: int` en factura y nota	UUID en mascotas, turnos, consultas, facturas y notas; el proveedor se identifica por CUIT	Evitar contadores globales que no sobreviven a la persistencia y asegurar que cada clase del sistema se pueda identificar de forma única
Administrador	Uno por clínica (1 a 1)	Lista de administradores; todos reciben las alertas de stock	Más flexible; sin impacto en las reglas
ABM	Alta de personal y catálogos	Alta, baja y modificación en `ControladorAdministracion`, con validaciones (documento/matrícula duplicados, un bloque horario por día, lotes no vencidos, proveedor con productos no se borra, baja de veterinario solo sin turnos futuros)	Cumplir CU-26 a CU-30 completos

Pruebas y resultados

Se escribieron **39 pruebas unitarias** y **19 casos funcionales** que recorren los casos de uso principales a través de los controladores (detalle en `docs/plan-pruebas.pdf`).

Ejecución	Pasan	Fallan
Código entregado (commit `8bf79b3`, solo con D-01 resuelto para poder compilar)	54 de 58	4 (`U-27`, `U-32`, `U-33`, `PF-13`: defectos D-02 y D-03)
Con correcciones aplicadas	58 de 58	0

Casos funcionales cubiertos: registro de dueños y mascotas, transferencia, asignación (válida, superpuesta, fuera de horario, con seña pendiente), reprogramación, cancelación con y sin cargo, ausencias y seña, urgencias con desplazamiento, consulta completa (peso, tratamiento, stock, confirmación, alerta), vacunas (especie, edad, refuerzo, recordatorio), notas rectificatorias, facturación (plan de salud, efectivo, crédito), anulación, ABM de administración, reportes y persistencia.

Alcance y límites. Las pruebas automáticas no incorporan las ventanas Swing.

Dificultades encontradas

- **Pasar del diseño al código:** Varias decisiones del diagrama final resultaron poco prácticas al implementarlas y hubo que rediseñarlas.
- **Persistir los objetos:** La serialización de dueños, mascotas, historias clínicas, turnos e inventario con referencias cruzadas obligó a simplificar el repositorio a un único `EstadoSistema`.
- **Probar código que usa la hora actual:** `Agenda`, `GestorUrgencias` y `Producto` leen la fecha del sistema, lo que dificultó las pruebas.
- **Trabajo en equipo y comunicación:** La distribución del trabajo fue más difícil de lo esperado, ya que no siempre hubo una participación equilibrada ni un seguimiento adecuado de las tareas asignadas. La comunicación por WhatsApp tampoco resultó efectiva para coordinar avances y resolver dudas. Además, en algunos casos se entregaron implementaciones generadas con IA sin una revisión suficiente, lo que trasladó al resto del equipo la responsabilidad de detectar errores, corregir inconsistencias e integrar código que no siempre cumplía con lo esperado.

Distribución de tareas

Integrante	Etapa 1 (documentación y diseño)	Etapa 2 (código, pruebas, documentación)
Erik Berot	Diagrama y documento de casos de uso; documento final con todos los puntos (UML, PDF); armado de los pull requests de cada punto	Estructura del proyecto; los cinco controladores; persistencia (`Repositorio`, `EstadoSistema`); paquete `servicio`; vistas de consola y Swing, ventana principal y `Entrada`; datos de prueba; correcciones de errores; Creación y ejecución del Plan de Pruebas; `README.md` e informe final; integración de ramas (≈31 *commits*)
Matías Rosental	Análisis del enunciado (Punto 1); diagramas de secuencia	Clases del modelo, constructores y *getters*; ajustes de clases de inventario, personas y facturación (≈8 *commits*)
Máximo Rombolá	Diagrama de clases inicial y documentación; patrones aplicados	Esqueleto de clases del modelo; primeras vistas; pruebas iniciales; ajustes de controladores y servicios.(≈7 *commits*)

Reflexión del grupo

- **Diseñar antes de programar sirvió, pero el diseño no es definitivo:** Los casos de uso, las reglas y los supuestos numerados (S-01 a S-30) fueron la referencia constante, tanto para implementar como para escribir las pruebas. En cambio, varios detalles de las clases cambiaron al implementarlas, y documentar esos cambios es parte del trabajo.
- **Separar responsabilidades facilitó el cambio:** Poder tener una interfaz de consola y otra Swing sobre los mismos controladores, o cambiar la forma de persistir sin tocar el dominio, justificó los principios SOLID y los patrones GRASP aplicados en la Etapa 1.